

Informe de Desarrollo

Torneo de Sudoku: Validación y Análisis de Partidas (LFP Numerix)

Retos técnicos encontrados y soluciones aplicadas

1. Inconsistencia en el diseño de acceso a la matriz

Durante la construcción del validador surgió la necesidad de extraer filas, columnas y cajas de 3x3 tanto de la matriz del tablero original como de la matriz de la solución propuesta por el jugador. En un primer intento, los métodos `obtener_fila`, `obtener_columna` y `obtener_caja` se definieron dentro de la clase `Tablero`, pensando únicamente en el tablero original.

Al programar la función `validar_intento`, se hizo evidente que esos mismos métodos se necesitaban para leer `intento.matriz_solucion`, que no es un objeto `Tablero`, sino una matriz suelta generada dentro de la clase `Intento`. La primera solución que se consideró fue crear un objeto `Tablero` "temporal" solo para reutilizar esos métodos, pero se descartó por ser conceptualmente incorrecta: una solución de jugador no es un tablero del dominio del problema (no tiene dificultad, ni id, ni pistas propias).

Solución aplicada: se movieron `obtener_fila`, `obtener_columna` y `obtener_caja` fuera de la clase `Tablero` y se convirtieron en funciones independientes dentro de `Logica/validador.py`, que reciben una matriz genérica como parámetro en lugar de operar sobre `self`. Esto eliminó la duplicación de lógica y permitió que la misma función sirviera tanto para la matriz original como para la matriz de solución.

2. Error de índice por datos de entrada incompletos (`IndexError`)

Al probar la validación con datos reales, el sistema arrojó `IndexError: list index out of range` dentro de `verificar_pistas`, al intentar acceder a `matriz_original[fila][columna]`. La causa no fue un error de lógica en el código, sino que la cadena de 81 caracteres del tablero, copiada manualmente desde el enunciado, había perdido 3 caracteres durante el copiado, quedando en 78. Como `generar_matriz` recorre la cadena en bloques de 9 caracteres, una cadena más corta que 81 genera una matriz con menos de 9 filas completas, lo que provoca el desbordamiento de índice al recorrer las 9 filas esperadas.

Solución aplicada: se verificó la longitud de la cadena antes de usarla (`len(cadena) == 81`) y se sustituyó por una cadena validada de longitud exacta. Este hallazgo llevó a reforzar la importancia de comprobar la integridad de los datos de entrada antes de asumir que un error es de lógica.

3. Verificación de pistas como condición independiente de la validez matricial

El enunciado exige dos condiciones distintas para considerar un intento como resuelto correctamente: que el porcentaje de validez sea 100% (filas, columnas y cajas correctas) y que las celdas originales (pistas) no hayan sido alteradas. Estas dos verificaciones se implementaron como funciones separadas (`es_grupo_valido` para la validez matricial y `verificar_pistas` para las celdas fijas), ya que son lógicamente independientes: una solución puede tener las 27 agrupaciones correctas y, aun así, haber modificado una pista (por ejemplo, si el jugador resolvió un sudoku válido pero distinto al planteado). Separar ambas validaciones permitió detectar con precisión esta clase de error durante las pruebas, cuando un intento mostraba 100% de validez pero `resuelto_correctamente` seguía en `False`.

4. Datos de prueba no correspondientes entre archivos

Durante las pruebas del sistema completo, se generó una nueva confusión al reemplazar el tablero de sudokus.lfp sin actualizar la solución correspondiente en intentos.lfp, resultando en solicitudes de validación entre un sudoku y una solución que no correspondían al mismo rompecabezas. El sistema detectó correctamente la inconsistencia (marcando los intentos como no resueltos), lo que sirvió para confirmar que el motor de validación funciona como se espera incluso ante datos mal emparejados, y evidenció la importancia de mantener sincronizados los tres archivos de entrada durante las pruebas.

```
=====
TORNEO DE SUDOKU - NUMERIX
=====
1. Cargar archivo de sudokus
2. Cargar archivo de jugadores
3. Cargar archivo de intentos
4. Validar y calificar intentos
5. Generar Reporte: Resumen por Sudoku
6. Generar Reporte: Rendimiento por Jugador
7. Generar Reporte: Top 10 Mejores Tiempos
8. Salir
Seleccione una opcion: 1

reportes
└─ reporte_jugadores.html U
   reporte_sudokus.html U
   reporte_top10.html U
main.py
```

Resumen por Sudoku

ID	Dificultad	Cantidad de Intentos	Tiempo Promedio	Tasa de Exito
1	Facil	5	372.0 seg	60.0%
2	Media	5	313.0 seg	60.0%

Top 10 Mejores Tiempos

Posicion	Carnet	Nombre	Sudoku	Dificultad	Tiempo
1	202015678	Luis Ramirez	2	Media	150 seg
2	202012345	Ana Morales	1	Facil	198 seg
3	202019012	Andres Mendez	2	Media	220 seg
4	202014567	Maria Lopez	2	Media	275 seg
5	202011234	Diego Fuentes	1	Facil	342 seg
6	202018901	Paola Hernandez	1	Facil	410 seg

Rendimiento por Jugador

Carnet	Nombre	Nivel	Tableros Intentados	Validez Promedio	Tiempo Promedio	Resueltos Perfectamente
202011234	Diego Fuentes	Intermedio	1	100.0%	342.0 seg	1
202012345	Ana Morales	Experto	1	100.0%	198.0 seg	1
202013456	Carlos Pérez	Principiante	1	92.59%	410.0 seg	0
202014567	Maria Lopez	Intermedio	1	100.0%	275.0 seg	1
202015678	Luis Ramirez	Experto	1	100.0%	150.0 seg	1
202016789	Sofia Castillo	Principiante	1	88.89%	500.0 seg	0
202017890	Jorge Gonzalez	Intermedio	1	92.59%	320.0 seg	0
202018901	Paola Hernandez	Experto	1	100.0%	410.0 seg	1
202019012	Andres Mendez	Principiante	1	100.0%	220.0 seg	1
202010123	Valeria Ortiz	Intermedio	1	92.59%	600.0 seg	0